

# PLAN-CHANGES-22082026

---

## Plan — reconciliation with Target System, over a Kafka confirmation stream

---

Written 2026-08-22. This is a **plan for code & testing changes**, not a description of the current system. This file is the bridge from "recon proves the lane is internally consistent" to "recon proves Target System actually persisted what the loader sent" — the gap README Slide 7 and ARCHITECTURE.md's C3 (Recon Service) NOTWIRE subgraph have named since before this session and left unimplemented on purpose, pending exactly this work.

**Status 2026-08-23: landed and proven on GCP.** The GCP re-verification this plan listed as "the remaining verification step" is done, and the "What this loop does NOT verify" section below is now **superseded**: a full Composer DAG pass under `enable_kafka=true` ran all 8 tasks green with acceptance criterion 9 closing on *live* Managed Kafka confirmations (50/50 TARGET rows confirmed) — `docs/evidence/gcp-composer-20260823/` . Getting there took five fixes that only real infrastructure could surface (pod env vars, the Python `oauth_cb` contract, the Managed Kafka token encoding, kafka-clients on JDK 25, and the Cloud Run mock never being rebuilt by `make build-images` ); all five are in `docs/runbook-gcp.md` §B8. One honest gap remains, unchanged and unrelated: criterion 4's *optional* Kafka watermark check still cannot run from a laptop, because Managed Kafka has no public endpoint.

**Status 2026-08-22: landed.** All five build-sequence steps are done (see the checkmarks below). `make verify` is 9/9 green on a clean run; the negative path fires ( `make verify` reports 8/9 naming the unconfirmed `account_key` ); `ReconciliationMatcherTest` is green. The doc sweep is applied across README, ARCHITECTURE, apps/composer/terraform READMEs, runbook-gcp, production-readiness, tests/README, evidence-map. The GCP smoke/DAG re-verification under `enable_kafka=true` is the remaining verification step.

---

## Context — what is being added, and why

---

Three things, all pointed at closing that one named gap:

1. **Insertion into Target System stays exactly as it is today** — the Loader App `POST` s one JSON document per account to `target-system-mock` 's `/v1/accounts` API, keyed by `X-Idempotency-Key` . Nothing about the write path changes. This point is a **constraint** on the rest of the plan, not new work: the confirmation stream added below is a side-effect of that existing API accepting a write, not a replacement for it.
2. **Reconciliation is extended to read Target System's own confirmation of what it persisted, from a Kafka topic, as JSON messages** — the "streams of data... JSON or protobuf... Kafka topics" README

Slide 7 already promises. `target-system-mock` publishes one confirmation event per accepted account; `recon-service` gains a Kafka consumer that reads them for the run and joins them to `account_target` by `account_key`, exactly as ARCHITECTURE.md's `NOTWIRE` subgraph ( `Confirmation Consumer` → `Confirmation Decoder` → `Confirmation Matcher` ) already draws it.

3. **The join is tested against real send/confirm data, including the failure case** — not just "it runs", but "it catches an account we believe we sent that Target System never actually confirmed". That is the entire point of Slide 7 (*"that proof does not come from the loader's HTTP response"*), so the test suite has to prove recon would catch a confirmation that never arrives, not merely a confirmation that does.

## The one design question this raises, and how it resolves

**When does the mock publish, and what does the event carry?** The mock already holds the full posted document ( `accounts.put(accountId, doc)` ) at the moment it accepts a write (HTTP 201), and that document already carries `migration.runId` and `migration.dedupKey` (the account key, per `docs/PLAN-CHANGES-21082026.md` D2 — the name `dedupKey` itself is retained by that plan's explicit exception). So the mock needs no new input to publish a self-describing confirmation: `{runId, accountId, accountKey, confirmedAt}`.

**Decision: publish exactly once, synchronously, on 201 (a genuine new write) — never on a 200 duplicate-replay.** A duplicate is Target System re-confirming something it already confirmed; republishing would turn a clean 1:1 join into an ambiguous one for no reconciliation benefit. If the publish itself fails, the write is still acknowledged 201 (Target System *did* persist the account) but no confirmation follows — which is precisely the "sent but not confirmed" case point 3 above exists to catch, so this failure mode is a feature to test, not a bug to prevent.

**Why a topic scan and not a bounded consumer.** Recon runs once, after the load lane completes, and is explicitly "off the data path" ( `ReconService.java:48` ). The simplest correct read is: a fresh consumer group per invocation, `auto.offset.reset=earliest`, read to the end of the topic, filter to this `run_id`. That is not how a 20B-transaction production system would do it — a full scan per run does not scale — so this plan flags it explicitly in `docs/production-readiness.md` rather than pretending it is production-ready, the same way the plan that preceded this one flagged the count-then-DELETE idempotency mechanism and the harness's synthetic-only volume.

---

# Decisions

| #   | Decision                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Rationale                                                                                                                                                                                              |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RC1 | Insertion into Target System remains API-only ( <code>POST /v1/accounts</code> ). No change.                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Explicit constraint from the request; the confirmation stream is a side-effect of an accepted write, not an alternate write path.                                                                      |
| RC2 | <code>target-system-mock</code> publishes one JSON confirmation event per <b>accepted</b> (201) write, on a new Kafka topic, synchronously before the HTTP response is sent.                                                                                                                                                                                                                                                                                                                                                                                    | Matches Slide 7's "streams... consumed and matched against the load"; synchronous-before-response means a confirmation always corresponds to a response the loader actually saw.                       |
| RC3 | No confirmation is published for a 200 duplicate-replay.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | A duplicate is not a new persisted fact; republishing would make the join many-to-one for no reconciliation value.                                                                                     |
| RC4 | Confirmation event shape: <code>{"runId", "accountId", "accountKey", "confirmedAt"}</code> — JSON, one object per Kafka message, keyed by <code>accountKey</code> .                                                                                                                                                                                                                                                                                                                                                                                             | Everything the matcher needs to join to <code>account_target (run_id, account_key)</code> without recon re-deriving anything; keyed messages make per-key ordering and future compaction well-defined. |
| RC5 | New topic <code>target-system-confirmations</code> , separate from the existing <code>target-system-target</code> topic.                                                                                                                                                                                                                                                                                                                                                                                                                                        | The existing topic is the pipeline's <i>outbound</i> publish to Downstream Consumers (C1: `SYS -->                                                                                                     |
| RC6 | <code>recon-service</code> gains a Kafka consumer ( <code>org.apache.kafka:kafka-clients</code> , the first Kafka dependency in any Java module) that reads the confirmation topic once per run, filtered to <code>run_id</code> , and joins to <code>account_target</code> by <code>account_key</code> .                                                                                                                                                                                                                                                       | Direct implementation of the NOTWIRE subgraph's <code>Confirmation Consumer</code> → <code>Confirmation Decoder</code> → <code>Confirmation Matcher</code> chain already drawn in ARCHITECTURE.md.     |
| RC7 | The join adds one report section ( <code>targetSystemReconciliation</code> ) and one verdict clause: <b>any <code>account_target</code> row with no matching confirmation fails the run</b> . An unmatched confirmation (received, no corresponding <code>account_target</code> row) is reported but does not fail the run — it can only mean a stale message from a prior run's <code>account_key</code> collision, which the per-run <code>run_id</code> filter already prevents; keeping the count without failing on it is a diagnostic, not a false alarm. | This is the actual proof Slide 7 promises: "we believe we migrated N accounts" becomes a claim Target System itself corroborates, not one the platform asserts about itself.                           |
| RC8 | Acceptance criterion 9: "every TARGET row is confirmed by Target System" — <code>tests/acceptance.py</code> grows from 8 to 9 criteria.                                                                                                                                                                                                                                                                                                                                                                                                                         | The existing 8 check the lane is internally consistent; this is the one criterion that checks the external system agrees.                                                                              |
| RC9 | A new, explicit <b>negative</b> test proves the failure path: seed a document Target System accepts but whose confirmation is deliberately suppressed (via a mock-only test hook, not a code path that exists in production), and assert reconciliation fails the run and names the unconfirmed account.                                                                                                                                                                                                                                                        | Point 3 of the request. A reconciliation feature that has only ever been exercised against clean data has not been shown to catch anything; a suite                                                    |

| # | Decision | Rationale                                             |
|---|----------|-------------------------------------------------------|
|   |          | that never fails is not testing the failure detector. |

## Component map — every file touched, and how

### target-system-mock (Java) — the publisher

#### apps/target-system-mock/pom.xml

- Add `org.apache.kafka:kafka-clients` (version pinned via the root `pom.xml` `<dependencyManagement>`, mirroring how `apps/common` pins Jackson today).

#### apps/target-system-mock/src/main/java/ro/mig/vault/TargetSystemMock.java

- `main`: read `TARGET_SYSTEM_CONFIRMATION_BOOTSTRAP` (default `localhost:19092`, the local redpanda address, matching the existing `TARGET_SYSTEM_*` env-var convention) and `TARGET_SYSTEM_CONFIRMATION_TOPIC` (default `target-system-confirmations`); construct one `KafkaProducer<String, String>` at startup, close it on shutdown (a `Runtime.getRuntime().addShutdownHook`, matching no existing pattern in this file today — this is the first thing in the mock that owns a resource needing a clean shutdown).
- `handleAccounts`: after `accounts.put(accountId, doc); accepted.incrementAndGet();` and before `respond(exchange, 201, ...)`, build the confirmation JSON from `doc` (RC4) and `producer.send(new ProducerRecord<>(topic, accountKey, json))`. **Decision:** block on the send's future (small, local-scale prototype; real-volume async batching is a `docs/production-readiness.md` item, not this plan) so a publish failure is visible rather than silently dropped in a background callback.
- `handleStats`: add `confirmationsPublished` to the JSON stats payload, for the same reason `accepted / duplicatesIgnored` are already there — a live counter an operator or a test can read without touching Kafka directly.
- New **test-only** admin endpoint `POST /__admin/suppress-next-confirmation` — sets a one-shot flag; the next accepted write still returns 201 and stores the account, but skips the `producer.send`. This is RC9's hook: a way to manufacture the "sent but not confirmed" case deterministically, without ever disabling the confirmation publish in a real code path. Reset by `/__admin/reset` like every other piece of mutable state.

### recon-service (Java) — the consumer, matcher, and verdict

#### apps/recon-service/pom.xml

- Add `org.apache.kafka:kafka-clients` (same pinned version).

#### apps/recon-service/src/main/java/ro/mig/recon/ReconService.java

- New CLI args `--confirmation-bootstrap` / `--confirmation-topic` (defaults matching the mock's, same dash/underscore-tolerant parsing every other flag already uses).

- New method `readConfirmations(String runId) : a KafkaConsumer<String, String>` with a fresh, run-scoped `group.id` (e.g. `recon-<runId>`, so a re-run of the same `run_id` re-reads from the start rather than resuming a stale committed offset — matching the count-then-DELETE idempotency philosophy of "re-running a run id must be safe"), reads to the current end offset (`endOffsets` at the moment the consumer starts, not an unbounded `poll` loop — `recon` is off the data path and must terminate), decodes each JSON message, keeps only those whose `runId` matches.
- New method `matchConfirmations(targetKeys, confirmations) : set-based join on account_key — unconfirmed = targetKeys - confirmationKeys, unmatchedConfirmations = confirmationKeys - targetKeys`.
- `main` : call these after the existing `account_target` key query already run for criterion 5 (every key appears exactly once) — the same query result is reused, not re-issued.
- Report: add a `targetSystemReconciliation` object — `confirmationsRead`, `matched`, `unconfirmedTargetRows` (count + the actual `account_key` list, named the same way `record_lineage` names rather than merely counts, per the repo's existing convention), `unmatchedConfirmations` (count only — diagnostic per RC7). **Not implemented as written:** the report carries `confirmations`, `confirmedTargetRows`, `unconfirmedTargetRows` and `unconfirmedAccountKeys`, and no `unmatchedConfirmations` field. The count was dropped as redundant — the per-run `run_id` filter already excludes every confirmation that could be "unmatched", so the field would have been a constant zero dressed as a diagnostic.
- Verdict: a new failure branch — `if (!unconfirmed.isEmpty()) { ...exit 1... }` — alongside the existing balance/agreement/orphan checks, with a message naming the unconfirmed `account_key`s (bounded, e.g. first 20, same truncation style as nowhere else in this file today since no other list is ever large enough to need it — precedent set here for the first time).

`apps/recon-service/src/test/java/ro/mig/recon/ReconciliationMatcherTest.java` (new)

- Unit tests for `matchConfirmations` alone (no Kafka, no HTTP) — the set-difference logic RC9 depends on: a clean case (every key confirmed), an unconfirmed case (a target key with no confirmation), a stale-confirmation case (a confirmation for a key not in `targetKeys`, e.g. a different run), and a duplicate-confirmation case (same key twice — must not double-count as two matches).

## Html.java — the report's human-readable half

`apps/recon-service/src/main/java/ro/mig/recon/Html.java`

- New "Target System reconciliation" section, same table style as the existing "Source reconciliation" / "Transformation / load reconciliation" sections: `matched`, `unconfirmed` (highlighted red if non-zero, matching the existing verdict-banner color convention), `unmatched confirmations`.

## Terraform — the confirmation topic

`terraform/modules/kafka/main.tf`

- Generalise from one `topic / google_managed_kafka_topic` pair to a `topics` map variable (`{ target = "target-system-target", confirmations = "target-system-confirmations" }`), `for_each` over it for both the topic resource and the `bootstrap_servers / topic` outputs (which become a map output, e.g. `topics = {target = ..., confirmations = ...}`). **Decision:** keep one cluster

( `google_managed_kafka_cluster` ), two topics on it — a second Managed Kafka cluster would double the ~\$/vCPU-hour cost of the *already-off-by-default* Kafka switch for no isolation benefit at this scale.

#### `terraform/envs/dev/main.tf`

- Update the `module.kafka` call site for the new `topics` map shape; the `TARGET_SYSTEM_CONFIRMATION_TOPIC` / `TARGET_SYSTEM_CONFIRMATION_BOOTSTRAP` env vars the mock and recon read are added alongside the existing `KAFKA_BOOTSTRAP` / `KAFKA_TOPIC` in the DAG's `env_file` output and the Composer module's env vars ( `terraform/modules/composer/main.tf` , next to `MIG_TARGET_SYSTEM_URL` ).

## Local stack — the confirmation topic exists locally too

#### `local/scripts/init_infra.py`

- Create the confirmation topic ( `target-system-confirmations` , or the env-overridable `TARGET_SYSTEM_CONFIRMATION_TOPIC` ) alongside the existing `KAFKA_TOPIC` creation — same `AdminClient` / `NewTopic` call, looped over both topic names.

#### `.env.example`

- Add `TARGET_SYSTEM_CONFIRMATION_BOOTSTRAP=localhost:19092` and `TARGET_SYSTEM_CONFIRMATION_TOPIC=target-system-confirmations` under the existing "target system" section.

#### `local/docker-compose.yml`

- Pass the two new env vars to the `target-system-mock` service (alongside the existing `TARGET_SYSTEM_PORT` / `TARGET_SYSTEM_FAILURE_RATE` / `TARGET_SYSTEM_SEED` ), pointing at `redpanda:9092` (the in-network address, not the host-mapped `19092` ).

#### `local/scripts/run_pipeline.py`

- Pass `--confirmation-bootstrap` / `--confirmation-topic` to the `recon` invocation, reading the same two env vars ( `Config` in `pipelines/common/config.py` grows two fields, `target_system_confirmation_bootstrap` / `target_system_confirmation_topic` , following the existing `kafka_bootstrap` / `kafka_topic` naming pattern exactly).

#### `composer/dags/mig_000001_1.py`

- The `reconciliation` task's `java_app(...)` argument list grows the same two flags, sourced from the same two new Composer env vars.

## Tests — the acceptance criterion and the negative-path proof

#### `tests/acceptance.py`

- New check function `target_confirmed()` , criterion 9: query `reconciliation-report.json` 's `targetSystemReconciliation.unconfirmedTargetRows` (recon already wrote the report to GCS; acceptance reads it the same way criterion 3 reads `doc_json` rows — no new BigQuery/Kafka access

needed from the Python test, it trusts recon's own verdict, the same way criterion 1 trusts

`run_ledger.balanced` rather than re-deriving the equation).

- Docstring / module comment: 9 criteria, not 8.

**A new acceptance-level negative-path run** (RC9, point 3 of the request), added as a documented **manual** verification step in `docs/runbook-gcp.md` §A (not a `make verify` criterion — deliberately: `make verify` must stay a pass/fail gate against a *clean* run, never one that expects a specific run to already be broken):

```
curl -X POST $TARGET_SYSTEM_URL/__admin/suppress-next-confirmation
make run          # one account's confirmation is now deliberately missing
make verify       # criterion 9 must FAIL, naming the unconfirmed account_key
curl -X POST $TARGET_SYSTEM_URL/__admin/reset
```

This is the acceptance-level proof that mirrors `ReconciliationMatcherTest`'s unit-level proof — the same claim (recon-service actually fails on an unconfirmed account) checked at two altitudes, the pattern this repo already uses for the balancing equation itself (unit tests in `test_engine.py`, acceptance criterion 1 against the live stack).

---

## Build sequence (each step leaves `make verify` green)

---

1. **✓ Add the topic, wire the env vars, no behavior change.** Terraform `topics` map, `init_infra.py` second topic, `.env.example`, `docker-compose.yml`, `Config` fields. Nothing reads or writes the new topic yet. (Green — identical to today.)
2. **✓ The mock publishes.** `kafka-clients` dependency, producer wiring, confirmation event on 201, `confirmationsPublished` stat, the `/__admin/suppress-next-confirmation` test hook. Nothing consumes yet, so the topic simply accumulates messages no one reads. (Green — the mock's existing behavior toward the loader is unchanged.)
3. **✓ Recon consumes, matches, reports — but does not yet fail the run.** Kafka consumer, `matchConfirmations`, the new report section and HTML table, `ReconciliationMatcherTest`. The verdict clause is added but logs rather than exits non-zero, so a real gap (unlikely on a clean prototype, but not yet proven impossible) does not immediately break `make verify` while this lands. (Green — report grows a section, verdict unchanged.)
4. **✓ Recon fails the run on an unconfirmed account.** Flip the verdict clause from log-only to `System.exit(1)`. Run the manual negative-path check from the Tests section above to prove it actually fires, then reset and confirm a clean `make run` + `make verify` still passes 9/9. (Green — and now provably capable of going red.)
5. **✓ Acceptance criterion 9 + doc sweep** (below). (Green, 9 criteria.)

---

## Markdown updates

---

Every doc that currently states or implies "reconciliation with Target System is not wired" needs the opposite statement once step 4 above lands, cited against this file the same way `docs/PLAN-CHANGES-21082026.md` is cited today:



- **README.md Slide 7** — rewrite from "the production edge the prototype does not yet wire" to a description of what is wired: the confirmation topic, the join, the new failure mode. Slide 12's acceptance-criteria table grows a 9th row.
- **ARCHITECTURE.md**
  - The top-of-file status note gets a new paragraph (in the same style as the existing 2026-08-21 superseded note) pointing at this plan.
  - **C1** — the `VC -->|"confirmation / audit events..."| SYS` edge label and the accompanying "Reconciliation with Target System is a return edge" paragraph move from describing an unconsumed edge to describing a consumed one.
  - **C3 (Recon Service)** — the `NOTWIRE` dashed subgraph becomes a normal solid one: `Confirmation Consumer / Confirmation Decoder / Confirmation Matcher` get real source citations (`ReconService.java:<lines>`) in place of "not implemented"; the `-->|"...not wired"|` edges become solid `-->`; the "What this C3 exposes" bullets are rewritten from describing the gap to describing the mechanism (still noting the full-topic-scan-per-run scale caveat from RC-context above).
- **apps/README.md** — the `target-system-mock` bullet gains "publishes a confirmation event per accepted write"; the `recon-service` bullet gains "and cross-checks Target System's own confirmation stream"; a new row in the `common/` class table is not needed (no new shared class), but the "Local gotcha" section gains a note that `/__admin/reset` also clears the confirmation-suppression flag.
- **composer/README.md** — the Configuration table gains the two new env vars.
- **terraform/README.md** — wherever the Kafka module/topic is diagrammed or listed, reflect the two-topic shape.
- **docs/runbook-gcp.md** — the manual negative-path verification steps from the Tests section above; the "What is emulated" table's `Target System` row gains "+ publishes confirmations to redpanda locally / Managed Kafka on GCP".
- **docs/production-readiness.md** — new item: "recon's confirmation-topic read is a full scan per run, `auto.offset.reset=earliest` with a fresh consumer group every time; fine at prototype volume, needs a bounded/incremental read (e.g. seek-by- timestamp to the run's start, or a compacted topic keyed by `account_key` with recon reading only the compacted tail) before real volume." Cross-reference the existing §2 scale items (2.1 FileLoadsBigQueryWriter, 2.4 quota review) since this is the same category of "works at prototype scale, named explicitly as not yet real-scale-ready."
- **tests/README.md** — 9 criteria, not 8; the new criterion's one-line description; a short "How criterion 9 differs from 1–8" note (it is the only criterion that checks an *external* system's own claim, not an internal invariant).
- **docs/evidence-map.md** — the Kafka section gains the confirmation topic as a second thing to look for in the console tour.

---

## Verification

---

- `make verify` after step 5: 9/9, all green, on a clean run.
- The manual negative-path check (Tests section): criterion 9 fails, names the unconfirmed `account_key`, `make verify`'s summary line reports 8/9 (not 9/9) — proving the new criterion is load-bearing, not decorative.
- `mvn test: ReconciliationMatcherTest` green, alongside the existing Java suite.



- `terraform validate + terraform plan` (defaults): clean, no diff for the Kafka-disabled default path (VPC connector, Kafka IAM grants and mock SA are all gated on `enable_kafka`).
- `terraform plan -var=enable_kafka=true`: clean; the plan shows the Serverless VPC Access connector ( `module.vpc_connector` ), the three project-level `roles/managedkafka.client` grants (dataflow-worker, recon-service, target-system-mock — the provider exposes no cluster-level IAM resource, so the grant is project-level via `google_project_iam_member` ), the `target-system-mock` service account, and the `KAFKA_SECURITY_PROTOCOL=SASL_SSL` env on the mock.
- A minimal `terraform apply -var=enable_kafka=true` confirms the infra provisions (the broker, connector and IAM grants all create cleanly). Followed by a full `terraform destroy` — no billed resources left.

## What this loop does NOT verify (the host-reachability constraint)

**Superseded 2026-08-23.** Written before the Heavy loop ran. The paragraphs below correctly describe why `make smoke-gcp` cannot prove criterion 9 — and that is still true — but their conclusion ("a live criterion-9 green on GCP is out of scope for this loop") no longer holds: the full Composer DAG path was run, and criterion 9 is green on live confirmations. What survives is the narrower statement: **the host cannot reach VPC-internal Kafka**, so the laptop-side acceptance run reports criterion 4's Kafka half as unreachable. Kept as written because the reasoning about which execution paths sit inside the VPC is what made the Heavy run work.

A live criterion-9 green on GCP is **out of scope for this loop** and documented honestly. `make smoke-gcp` runs `recon-service` and `json_producer` on the **host laptop** (inside the `mig-toolbox` podman container), not in the VPC. Managed Kafka is VPC-internal (no public endpoint), and the Serverless VPC Access connector serves serverless GCP services (Cloud Run), **not a host laptop** — a VPC connector is not a VPN. So the host-side `recon/json_producer` **cannot reach VPC-internal Kafka at all**.

The only execution path where `recon` and `json_producer` run inside the VPC is the Composer DAG (pods on Composer's GKE cluster in `mig-vpc` ). A live criterion-9 green on GCP therefore requires the full Composer DAG path (~20-40 min Composer create, ~\$300-400/mo idle, RBAC via VM, 6 image rebuilds, DAG trigger, pre-staged extract, poll to green) — not `make smoke-gcp`.

This loop lands the **code** (OAUTHBEARER callback handlers in Java and Python, the mock SA, the `KAFKA_SECURITY_PROTOCOL` switch), the **infra** (VPC connector module, project-level Kafka IAM, Cloud Run v2 `vpc_access` wiring), and the **plan/apply validation** (the broker, connector and grants all provision cleanly). The mock publishes from Cloud Run, which is inside the VPC, so its path is wired correctly; only the host-side `recon` consume is DAG-only, and that is the path a future Heavy run takes. See `docs/runbook-gcp.md` §B8.

## Out of scope

- **Protobuf confirmations.** README Slide 7 says "JSON or protobuf"; this plan implements JSON only, since the mock is the producer and controls its own format — protobuf would require a shared `.proto` contract with no second party to negotiate it against in this prototype. Noted as a future format option, not a gap.

- **"Late" confirmations.** ARCHITECTURE.md's NOTWIRE subgraph names "unmatched / late / duplicate" as the three failure modes to flag. This plan implements unmatched (RC7's unconfirmedTargetRows ) and duplicate ( ReconciliationMatcherTest 's duplicate-confirmation case, folded into "matched" since a duplicate confirmation for an already-matched key is not itself a problem) but not "late" — recon runs once, synchronously after the load lane, so there is no fixed deadline a confirmation could arrive after. A time-boxed recon (retry until a deadline, then declare stragglers "late" rather than merely "not yet seen") is real production behavior this prototype does not need.
- **Scaling the confirmation read past a full topic scan.** Named explicitly in the docs/production-readiness.md addition above; not solved here.
- **Collapsing the two orchestrators' now-slightly-longer task lists.** Already tracked as out of scope in docs/PLAN-CHANGES-21082026.md ; this plan adds two CLI flags to each orchestrator's reconciliation call, which does not change that item's shape or urgency.